Assignment 2: Paper Summarization and Reproduction

HyperImpute: Generalized Iterative Imputation with Automatic Model Selection
Jarrett, Cebere, Liu, Curth, van der Schaar (ICML 2022)

Yann Klees 0800619
Data Wrangling, Utrecht University

24 October 2026

Part 1: Paper Summary

1. Main Problem

Each existing family of imputation methods has a strength that the other lacks. Iterative imputers such as MICE and MissForest model each column flexibly and handle incomplete data when values are missing at random (MAR), meaning that missingness depends only on observed values. Each column model, however, must be chosen and tuned by hand. Deep generative imputers such as GAIN and MIWAE need no manual tuning, but they are hard to train, and most require complete training data or assume that values are missing completely at random (MCAR). HyperImpute aims to combine flexible per column modelling, automatic configuration and training on incomplete data under MAR.

2. Proposed Solution

HyperImpute keeps the column by column structure of iterative imputation. Instead of using one fixed model chosen by the user, it searches automatically over several model classes and their hyperparameters for each column (Section 3.2). Algorithm 1 (Section 3.4) runs in four steps.

1. Fill every missing cell with a simple first guess, by default the column mean.
2. Visit the columns one at a time. For each column, train on the rows where its value is observed, using all other columns, including their current guesses, as inputs.
3. Try several learners and hyperparameters, keep the one that predicts the column best, and use it to fill the rows where the column is missing.
4. Repeat over all columns until the imputations stop changing.

The candidate pool contains linear and logistic regression, random forests, XGBoost, CatBoost and small neural networks, with regressors for continuous columns and classifiers for categorical ones (Section 4, Appendix A.3). Without the search, each column gets a fixed model and the algorithm reduces to classic iterative imputation, so HyperImpute generalises it.

3. Optimization Techniques and Special Cases

Section 3.3 treats the position of the model search relative to the imputation loop as the key design choice. A top down search reruns the whole imputation for every combination of column models, which with D columns and A configurations each means A^D runs and is far too expensive. A concurrent search tunes each column on its own at a cost of $O(KDA)$ for K rounds, but it ignores how one column's model affects the others. HyperImpute reverses the order: the imputation loop runs outside, and a small search for each column runs inside it. The cost stays at $O(KDA)$, yet every search uses the latest imputations of all other columns.

By default, the search uses an adapted Hyperband (Section 4, Appendix A.4). Hyperband trains many candidates briefly, drops the weak ones and gives the survivors more training. Because neural networks count training in epochs and forests in trees, both are converted into one common budget.

Naive search and Bayesian optimisation are also supported. Cross validation scores each candidate, with RMSE for continuous and AUROC for categorical columns. Later rounds can skip the search and reuse earlier choices. The loop stops after a maximum number of rounds, when the imputations barely change, or when the objective stops improving (Appendix A.5).

4. Strengths and Limitations

Strengths.

- It removes the manual model choice that limits iterative imputation, while its theory still covers MCAR and MAR.
- Unlike most deep generative imputers, it trains on incomplete data and handles mixed data types (Table 1).
- An ablation (Table 2) attributes a performance gain to each design component.

Limitations.

- The separate column models are not guaranteed to form a valid joint distribution over all variables.
- It produces a single imputation, so it does not quantify uncertainty about the filled values.
- MNAR is outside the paper's scope, and no theory is given for it.

5. Evaluation Methodology

The authors test HyperImpute on 12 UCI datasets with artificially removed values, so the true values are known. They simulate MCAR, MAR and MNAR at rates from 10% to 70%, with MAR at 30% as the main setting, and compare against eight baselines including MICE, MissForest, GAIN and MIWAE. Errors are measured with RMSE and the Wasserstein distance (WD), averaged over 10 seeds.

HyperImpute is best on both metrics on 10 of the 12 datasets and wins at least one metric on all of them. In the ablation, it lowers RMSE by 18% on average compared with the best single learner variant. Simple models are selected on small datasets and at high missingness, while boosting and neural networks win on large ones. The imputations stabilise after about four rounds.

Part 2: Reproduction

Code: `https://github.com/YannKleees/hyperreimpute`. I use the authors' `hyperimpute` package [2], version 0.1.17, with the settings of their notebook `experiments_04_gain_of_function.ipynb`. All runs were on CPU without a GPU.

```
git clone https://github.com/YannKleees/hyperreimpute.git && cd hyperreimpute
pip install -r requirements.txt
python experiments.py        # writes results/*.csv, then open reproduction.ipynb
```

1. Selected Experiment

I reproduce the source of gains ablation (Table 2), which isolates the paper's central claim that per column model selection improves iterative imputation. It compares single learner ICE, per column selection with default hyperparameters and full HyperImpute, under MAR with 30% missingness and 10 seeds. I evaluate airfoil, compression and wine_white; california and letter were computationally infeasible on a laptop. I add one control, ice_cb_fixed (Section 4).

2. Comparison with the Paper

Table 1. RMSE × 10 on the masked cells under MAR with 30% missingness, averaged over 10 seeds. Lower is better.

	airfoil		compression		wine_white	
Method	Paper	Ours	Paper	Ours	Paper	Ours
ice_lr	2.35	1.95	1.43	1.49	0.92	0.86
ice_rf	2.37	1.91	1.62	1.68	0.99	0.90
ice_cb	2.08	2.55	1.25	2.29	0.88	1.10
column_naive	1.86	1.36	1.09	1.15	0.85	0.79
HyperImpute	1.48	1.80	1.01	1.48	0.80	0.79
ice_cb_fixed	–	**1.25**	–	**1.04**	–	**0.75**

The paper reports standard deviations between 0.3 and 0.5 on airfoil and between 0.03 and 0.22 on the other two datasets. My `column_naive` runs HyperImpute without hyperparameter search, so unlike the paper's version it can still switch models between iterations.

The baselines reproduce well. `ice_lr`, `ice_rf` and `column_naive` lie within about one reported standard deviation on all three datasets. The only exception is `ice_rf` on wine_white, where the reported standard deviation is just 0.03. The central mechanism of the paper therefore holds, since per column model selection lowers RMSE by 8% to 29% relative to the best single learner.

Full HyperImpute reproduces only on wine_white. On airfoil and compression its RMSE exceeds the reported value by 22% and 46%, and its Wasserstein distance is about 2.7 times higher, at 0.79 and 0.81 against 0.29. Hyperparameter tuning also makes HyperImpute worse than `column_naive`, by 32% on airfoil and 28% on compression, while raising the runtime from about 30 seconds to between 110 and 450 seconds per run. `ice_cb` deviates strongly on every dataset. A correctly configured CatBoost, `ice_cb_fixed`, achieves the lowest RMSE everywhere and improves on `column_naive` by 6% to 10%.

3. Parameters

Table 2. Parameters used in all runs.

Parameter	Value
Setup	
Software	`hyperimpute` 0.1.17, pandas below 2.3, Python 3.11, CPU only
Datasets	airfoil, compression, wine_white, min max scaled
Missingness	MAR, rate 0.3, `column_limit=10`, seeds 0 to 9
Metrics	RMSE on masked cells, Wasserstein distance summed over columns
Methods	
HyperImpute	`optimizer="hyperband"`, `select_lazy=False`
column_naive	`optimizer="simple"`
ICE variants	`optimizer="simple"`, `select_model_by_column=False`, `select_model_by_iteration=False`, one fixed learner
ice_cb_fixed	ICE with CatBoost, `n_estimators=1000`, `learning_rate=0.03`
Package defaults	
Learner pool	linear and logistic regression, random forest, XGBoost, CatBoost
Imputation	mean initialisation, random column order
Search	`n_inner_iter=40`, `select_patience=5`, `optimize_thresh=5000`

All other settings keep the package defaults. Neural networks join the learner pool only when a GPU is available, so they were not used here.

4. Reasons for the Differences

The single learner baselines reproduce, so the data and masking are not the cause. The most likely cause is that the package changed after publication. In version 0.1.17, CatBoost defaults to 10 trees with a learning rate of 0.001, so its predictions barely move away from the column mean. This explains `ice_cb`. It cannot explain the HyperImpute gap, because `column_naive` uses the same defaults and still reproduces. The difference lies in the search. Hyperband tunes CatBoost only between 10 and 100 trees and at learning rates from 0.01 to 0.04, a range added after version 0.1.5, which was released with the paper. The tuned search therefore offers weaker candidates than the paper's, and as it also scores them on the observed rows only, it can prefer configurations that do worse on the missing ones. With 1000 trees and a learning rate of 0.03, CatBoost performs well again, as `ice_cb_fixed` shows. The missing neural networks and seed variation play a smaller role. Runtimes come from two parallel jobs on one CPU, so only their ratios are meaningful.

5. Conclusion

The central claim reproduces, since per column model selection clearly outperforms iterative imputation with a single linear or forest model. The reported numbers for full HyperImpute reproduce only on wine_white, most likely because later package versions restricted the CatBoost search range. Reproductions of this paper should therefore fix the package version.

References

[1] D. Jarrett, B. Cebere, T. Liu, A. Curth, and M. van der Schaar. HyperImpute: Generalized Iterative Imputation with Automatic Model Selection. In *Proceedings of the 39th International Conference on Machine Learning*, PMLR 162:9916–9937, 2022.

[2] HyperImpute source code. `https://github.com/vanderschaarlab/hyperimpute`